

03

깃과 브랜치

03-1 브랜치란

03-2 브랜치 만들기 및 이동하기

03-3 브랜치 정보 확인하기

03-4 브랜치 병합하기



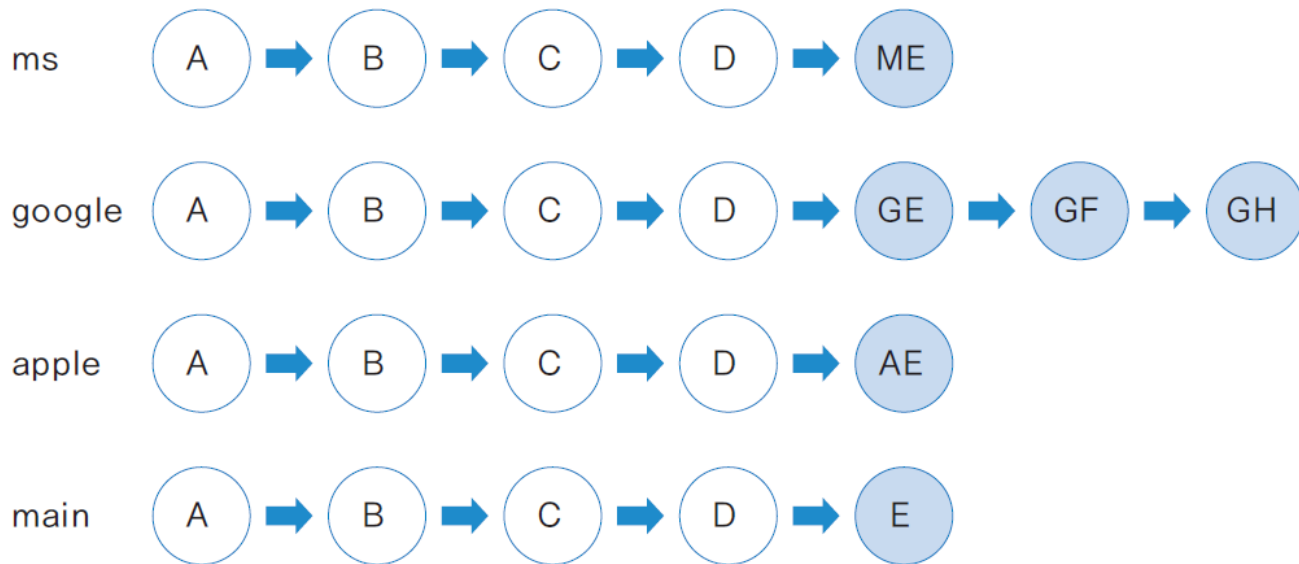
브랜치가 필요한 이유 - (1)

- 예를 들어 우리가 어떤 제품의 사용 설명서를 만들고 버전 관리는 깃으로 한다고 상상
제품이 출시되기 전에는 다음 그림처럼 개발 순서에 따라 사용 설명서를 작성
하지만 제품이 출시되고 나면 문제가 생기는데 고객사마다 추가로 요구하는 내용이 달라질 수 있기 때문
- 이렇게 고객사의 요구 사항을 반영하다 보면 제품이 달라질 것이고, 그에 따라 사용 설명서도 달라져야 함



브랜치가 필요한 이유 - (2)

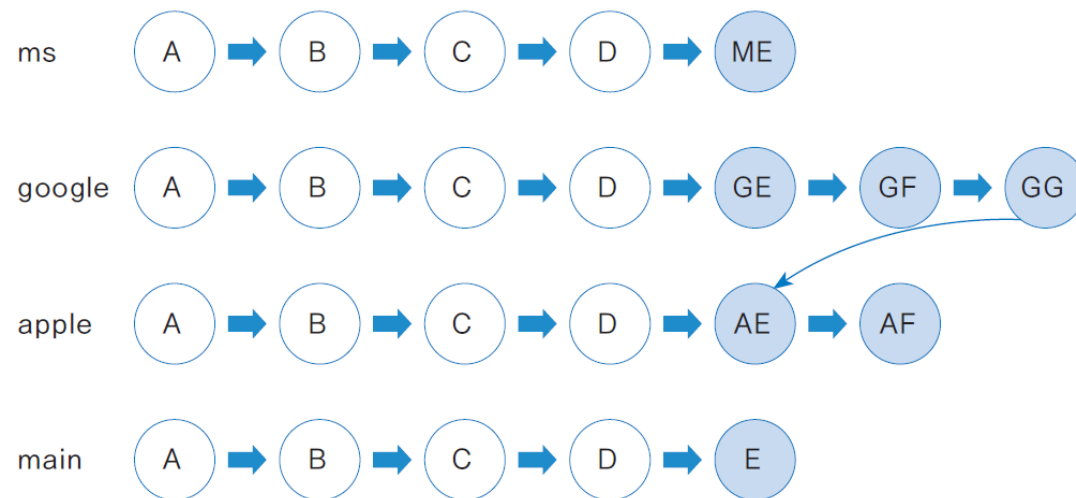
- 먼저 떠오르는 해결책은 처음에 작업했던 저장소(다음 그림에서 main) 전체를 여러 개 복사해서
고객사(다음 그림에서 apple, google, ms)마다 이름을 붙인 후 저장소마다 버전 관리를 따로 하는 방법임
- 하지만 이 방법은 비효율적인데 먼저 고객사마다 디렉토리를 복사하면
출시 전까지 만들었던 내용은 동일하기 때문에 자료가 중복됨
- 버전 관리 시스템의 장점은 파일 이름을 더럽히지 않는 것인데, 이 방법은 고객사마다 디렉터리 이름을 다르게 사용해야 함



저장소마다 버전 관리를 따로 하는 방법

브랜치가 필요한 이유 - (3)

- google에서 작업을 마쳤는데 나중에 apple에서도 필요한 내용이라고 가정하면 google에 있는 최신 상태의 코드를 복사해서 apple 디렉터리에 붙여 넣은 다음, apple 디렉터리에서 새로운 버전을 커밋하면 문제가 생길 수 있음
- GG 버전에서 필요한 부분의 코드만 복사해서 붙여 넣으려 해도 GE와 GF 버전의 내용이 AE 버전에는 반영되어 있지 않으므로 오류가 날 수 있으며 그렇다고 GG 버전의 코드 전체를 그대로 덮어 버리면 apple의 D 버전에서 AE 버전을 만들면서 수정한 내용이 의도치 않게 바뀌거나 사라짐
- 이런 **지옥에서 우리를 구원해 주는 도구가 바로 브랜치**임



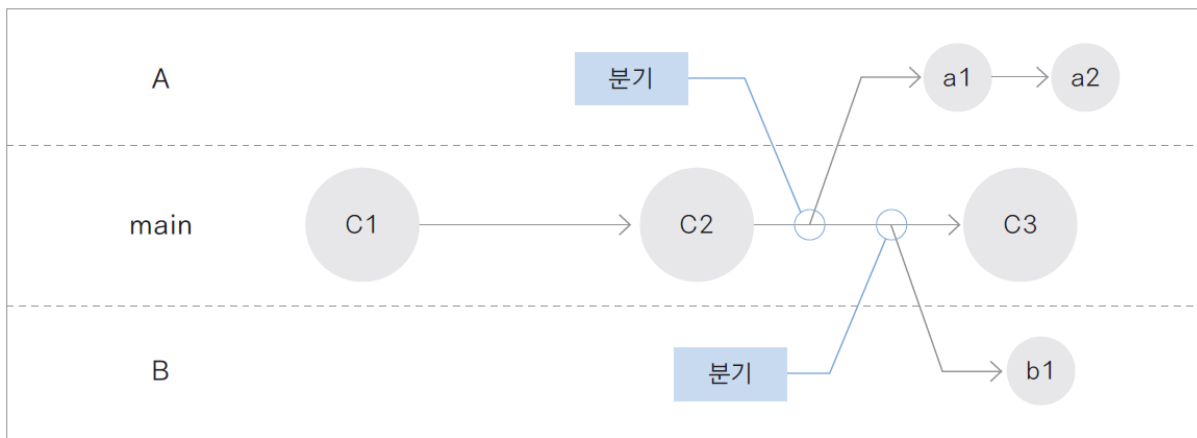
google 디렉터리의 최신 버전 코드를 복사해 apple 디렉터리에 붙여 넣고 커밋하는 과정

master 브랜치? main 브랜치!

- 깃으로 버전 관리를 시작하면 기본적으로 **main이라는 브랜치**가 만들어짐
- 옛 버전의 깃을 사용하고 있거나, 최신 깃을 설치했더라도 메인 브랜치를 main으로 지정하지 않았다면 master 브랜치로 나타날 수도 있음
- main 브랜치와 master 브랜치는 이름만 서로 다를 뿐, 깃을 사용할 때 기본이 되는 브랜치라는 개념은 같음
- master라는 이름이 master(주인)-slave(종)라는 노예 제도를 연상시키고 인종을 차별하는 느낌을 주기 때문에 main으로 바뀌었음
- 이미 깃을 사용해 왔다면 브랜치 이름은 master로 되어 있을 것이고 main 브랜치와 master 브랜치가 뒤섞여 헷갈릴 수도 있음
- 예전 작업은 master 브랜치를 사용하고 최근 작업은 main 브랜치를 사용하겠지만 시간이 흐르면 main 브랜치가 자연스러워질 것

브랜치 기능 살펴보기 - (1)

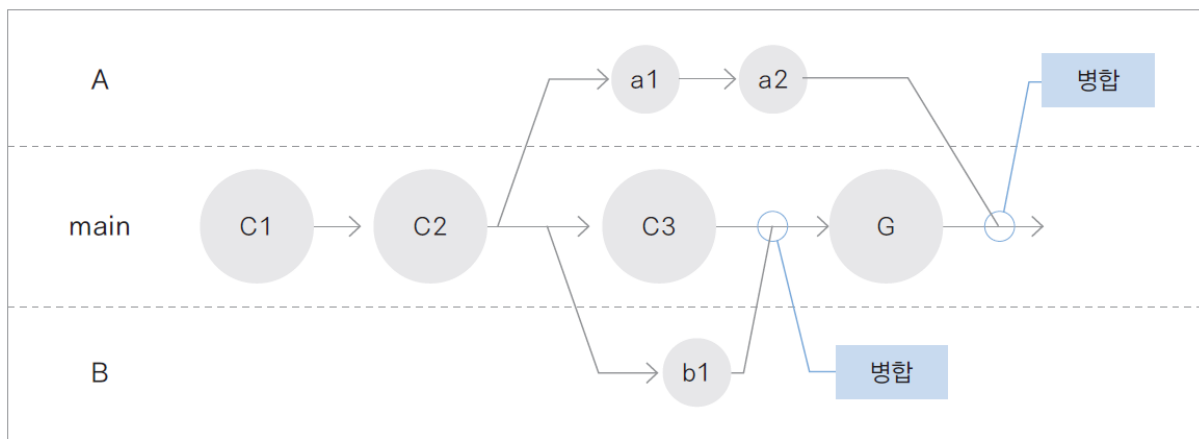
- 깃으로 버전 관리를 시작하면 기본적으로 main 브랜치가 만들어짐
- 사용자가 커밋할 때마다 main 브랜치는 어떤 게 최신 커밋인지 정보를 가짐
- 즉, 브랜치는 커밋을 가리키는 포인터와 비슷하다고 생각하면 됨
- 기존 파일은 main 브랜치에 그대로 유지하면서, 새 브랜치에서 기존 파일 내용을 수정하거나 새로운 기능을 추가할 수 있음
- 이렇게 main 브랜치에서 새 브랜치를 만드는 것을 '분기(branch)한다'라고 함



브랜치 분기 과정

브랜치 기능 살펴보기 - (2)

- 새 브랜치에서 원하는 작업을 다 끝냈다면 새 브랜치에 있던 파일을 원래 main 브랜치에 합칠 수 있음
이렇게 분기했던 브랜치를 main 브랜치에 합치는 것을 '병합(merge)'한다고 함
- 즉, 기본 파일은 main 브랜치에 그대로 둔 상태에서 새로운 브랜치를 분기한 후
수정이나 추가 작업을 하고 별 문제없이 완성되었다면, 새 브랜치의 내용을 main 브랜치와 병합하는 것



브랜치 병합 과정

실습 상황 만들기 - (1)

1. 터미널 창을 열어 홈 디렉터리에 'manual'이라는 새 디렉터를 만들고 해당 디렉터리로 이동

```
$ cd ~  
$ mkdir manual  
$ cd manual
```

2. manual 디렉터를 저장소로 만들고 ls -al 명령을 사용해서 .git 디렉터리가 만들어졌는지 확인

```
$ git init  
$ ls -al
```

3. manual 디렉터리 안에 work.txt 파일을 만들기 위해 터미널에 다음과 같이 입력해서 vim을 열고, 'content 1'이라는 내용을 입력한 후 저장

```
$ vim work.txt
```


실습 상황 만들기 - (2)

- 이제 work.txt 파일을 스테이지에 올리고 커밋함
커밋 메시지는 간단히 'work 1'이라고 함

```
$ git add work.txt  
$ git commit -m "work 1"
```

- 커밋이 완료되면 git log를 입력해서 커밋 내역을 확인
커밋 해시 오른쪽에 있는 (HEAD -> main)은 HEAD가 현재 main이라는 브랜치를 가리키고 있다는 뜻이고
(HEAD -> main)이 붙은 커밋이 가장 최신 커밋
현재는 work 1이라는 커밋이 '최신 커밋'이라고 알려 주고 있음

실습 상황 만들기 - (3)

6. work.txt 파일에 'content 2'를 추가한 후 'work 2'라는 메시지와 함께 커밋

```
$ vim work.txt    ← work.txt에 'content 2' 추가 입력 후 저장  
$ git commit -am "work 2"
```

7. 다시 한번 work.txt 파일을 열고 'content 3'을 추가한 후 'work 3'이라는 메시지와 함께 커밋
여기까지 초기 사용 설명서를 만들고 커밋까지 마칩

```
$ vim work.txt    ← work.txt에 'content 3' 추가 입력 후 저장  
$ git commit -am "work 3"
```

8. 마지막으로 커밋 로그를 다시 한번 더 확인하기 위해 git log를 입력
3개의 커밋이 만들어졌고, 가장 최신 커밋인 work 3에 (HEAD -> main)가 표시되어 있음

새 브랜치 만들기 - (1)

1. 깃에서 브랜치를 만들거나 확인하는 명령은 git branch

```
$ git branch
```

2. main이라고 나타남

저장소를 만들면 main 브랜치가 기본으로 생성되고, 그동안 main 브랜치에서 작업하고 있었던 것

3. 새로운 브랜치를 만들려면 git branch 명령 다음에 만들려는 브랜치 이름을 적음
고객사인 apple의 브랜치를 만들기 위해 다음과 같이 입력

```
$ git branch apple
```

4. apple 브랜치를 만들었는데 화면에는 아무것도 나타나지 않음
현재 저장소의 브랜치를 확인하기 위해 다음과 같이 입력

```
$ git branch
```

새 브랜치 만들기 - (2)

5. main 브랜치 위에 apple 브랜치가 추가된 것을 확인할 수 있음
main 앞에 * 가 표시된 것은 여러 브랜치 가운데 현재 작업하는 브랜치를 나타냄
아직 main 브랜치에서 작업하고 있다는 뜻
6. 브랜치가 추가된 후에는 커밋 로그 화면도 다르게 나타남
git log 명령을 사용해 커밋 로그를 확인해 보면 그동안 커밋 로그를 확인할 때 (HEAD -> main)라고 표시되던 곳에 apple 브랜치가 추가되면서 (HEAD -> main, apple)로 바뀌어 있음
저장소에 main과 apple이라는 2개의 브랜치가 있고, HEAD -> main이므로 현재 작업하는 브랜치는 main 브랜치라는 의미
7. 앞에서 소개한 방법으로 google 브랜치와 ms 브랜치를 만들고 git branch 명령으로 저장소 안에 있는 모든 브랜치를 확인

```
$ git branch google  
$ git branch ms  
$ git branch
```

새 커밋을 추가하면 어떻게 될까?

1. git log로 확인해 보면 work 3 커밋 해시의 오른쪽에 main 브랜치를 비롯해 ms, google, apple 브랜치가 함께 표시되어 있음
즉, 이것은 main 브랜치뿐만 아니라 ms와 google, apple 브랜치에도 최신 커밋이 work 3이라는 뜻

2. work.txt 파일 안에 'main content 4'라는 내용을 추가한 후 'main work 4'라는 메시지와 함께 커밋

```
$ vim work.txt    ← work.txt에 'main content 4' 추가 입력 후 저장  
$ git commit -am "main work 4"
```

3. git log 명령에 --oneline 옵션을 추가해서 확인
--oneline 옵션은 한 줄에 한 커밋씩 보여주기 때문에 여러 커밋을 한눈에 확인할 때 편리

```
$ git log --oneline
```

4. 최신 커밋인 main work 4는 main 브랜치에만 적용되어 있고 ms와 google, apple 브랜치는 아직 work 3 커밋 상태
즉, 새로 만든 커밋은 현재 브랜치(main 브랜치)에만 적용되고 나머지 브랜치에는 적용되지 않음
브랜치별로 커밋을 따로 관리할 수 있음

브랜치 전환하기 — git switch - (1)

1. 현재 브랜치에서 다른 브랜치로 전환하려면 git switch 명령 다음에 브랜치 이름을 사용
다음과 같이 입력해서 apple 브랜치로 전환

```
$ git switch apple
```

2. \$ 커서 바로 위에 나타난 파일 경로 끝에 (apple)이라고 표시됨
현재 apple 브랜치에서 작업하고 있다는 뜻
3. 작업 브랜치를 apple 브랜치로 바꿨을 때 어떤 변화가 생기는지 확인
git log 명령을 사용해서 현재 브랜치의 커밋 로그를 확인

```
$ git log --oneline
```

4. apple 브랜치를 만들기 전까지 main 브랜치에 있던 커밋들은
그대로 apple 브랜치에도 적용되었으므로 work 3 커밋까지 표시됨
커밋 해시에서 HEAD가 apple을 가리키는 것도 확인할 수 있음

브랜치 전환하기 — git switch – (2)

5. cat 명령을 사용해 work.txt 파일의 내용을 확인

```
$ cat work.txt
```

6. 최신 커밋이 work 3이기 때문에 content 1부터 content 3까지 3개의 행만 있음
main 브랜치에서 입력했던 main content 4가 없음
main content 4는 apple 브랜치를 분기한 후에 main 브랜치에 추가된 커밋이기 때문에
apple 브랜치에 영향을 미치지 않았다는 것을 알 수 있음

전환한 브랜치에서 커밋하기 - (1)

1. apple 브랜치에는 브랜치를 분기할 때 가져온 work.txt 파일이 있음
빔에서 이 파일을 열고 'apple content 4'라는 텍스트를 추가하고 저장

```
$ vim work.txt
```

 ← work.txt에 'apple content 4' 추가 입력 후 저장

2. apple 고객사만을 위한 내용이 담긴 apple.txt 문서를 만들고 여기에도 똑같이 apple content 4를 입력하고 저장

```
$ vim apple.txt
```

 ← apple.txt를 만들고 'apple content 4' 입력 후 저장

3. 수정한 work.txt 파일과 새로 만든 apple.txt 파일을 각각 커밋할 수도 있지만, 여기에서는 2개를 묶어서 한꺼번에 커밋하려고 함
이럴 때 git add 명령 뒤에 한 칸 땀 후 마침표(.)를 붙이면
현재 저장소에서 수정 내용이 있는 파일을 스테이지에 한꺼번에 올릴 수 있음
다음 명령을 실행하고 'apple work 4'라는 메시지와 함께 커밋

```
$ git add .  
$ git commit -m "apple work 4"
```


전환한 브랜치에서 커밋하기 - (2)

4. git log 명령을 사용해 커밋이 어떻게 저장되었는지 확인

```
$ git log --oneline
```

5. 첫 번째 줄에 (HEAD -> apple)이라고 되어 있으니 현재 apple 브랜치로 전환한 상태
apple 브랜치의 최신 커밋이 apple work 4라는 걸 알 수 있음

브랜치와 커밋 관계 알아보기 - (1)

1. git log 명령을 사용할 때 --branches 옵션을 추가하면 브랜치마다 최신 커밋을 한눈에 살펴볼 수 있음
이렇게 하면 main 브랜치에는 없고 apple 브랜치에만 있는 커밋, 즉 apple content 4 커밋을 보여줌

```
$ git log --oneline --branches
```

2. 결과 화면을 보면 커밋 해시마다 오른쪽에 (HEAD -> apple), (main), (ms, google)이라고 표시되어 있음
이 부분을 보면 해당 커밋이 어떤 브랜치에서 만든 것인지 구별할 수 있음
(HEAD -> apple)이라고 되어 있으니 현재 브랜치는 apple 브랜치고, 최신 커밋은 apple work 4
main 브랜치의 최신 커밋은 main work 4이고, ms 브랜치와 google 브랜치의 최신 커밋은 work 3

3. 브랜치와 커밋 관계를 좀 더 보기 쉽게 그래프 형태로 표시하려면 git log 명령에 --graph 옵션을 추가하면 됨

```
$ git log --oneline --branches --graph
```

브랜치와 커밋 관계 알아보기 - (2)

4. 커밋 내역 왼쪽에 수직선(|)이 커밋과 커밋의 관계를 보여 줌
그래프를 살펴보면 apple 브랜치의 최신 커밋은 apple work 4인데, 점선을 따라 apple work 4 커밋의 부모를 찾아가 보면 work 3 커밋을 만나게 됨
즉, apple 브랜치에서는 work 3 커밋 다음에 apple work4 커밋이 만들어졌다는 뜻
5. main 브랜치의 최신 커밋은 main work 4이고 대각선을 따라가 부모 커밋을 찾아보면 work 3 커밋임
apple 브랜치의 커밋과 main 브랜치의 커밋이 같은 부모 커밋을 가지고 있음
즉, main 브랜치나 apple 브랜치는 work 3 커밋까지는 같고 그 이후부터 브랜치마다 다른 커밋을 만들었다는 사실을 알 수 있음

브랜치 사이의 차이점 살펴보기

- 브랜치마다 커밋이 점점 쌓여 갈수록 브랜치 사이에 어떤 차이가 있는지 일일이 확인하기 어려워짐
- `git log` 명령에서 브랜치 이름 사이에 마침표 2개(..)를 넣는 명령으로 브랜치 간의 차이를 쉽게 확인할 수 있음
- 브랜치 이름과 마침표 사이는 공백 없이 입력하며, 마침표 왼쪽에 있는 브랜치를 기준으로 오른쪽 브랜치와 비교

- 예를 들어 `main` 브랜치와 `apple` 브랜치 사이의 차이점이 무엇인지 확인하려면 다음과 같이 입력

```
$ git log main..apple
```

- 이렇게 하면 `main` 브랜치에는 없고 `apple` 브랜치에만 있는 커밋 즉, 여기에서는 `apple` work 4 커밋을 보여 줌
- 반대로 `apple` 브랜치를 기준으로 `main`와 비교하면 `apple` 브랜치에는 없고 `main` 브랜치에만 있는 `main` work 4 커밋을 보여 줌

```
$ git log apple..main
```

서로 다른 파일 병합하기 - (1)

1. 터미널 창을 열고 홈 디렉터리로 이동

'manual-2'라는 디렉터리를 만들고 깃 저장소를 만들 텐데, 2가지 과정을 한꺼번에 처리

git init 다음에 디렉터리 이름을 입력하면 새로운 디렉터리를 만들고 저장소를 초기화하는 과정을 한꺼번에 처리할 수 있음

```
$ cd ~  
$ git init manual-2
```

2. manual-2 디렉터리로 이동한 후 ls -al 명령을 사용해서 .git/ 디렉터리가 만들어졌는지 확인

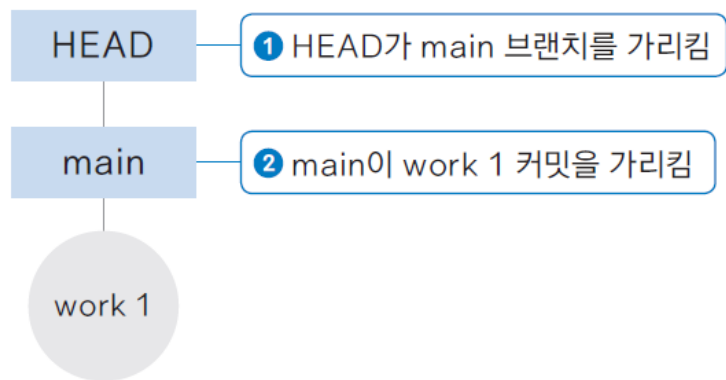
```
$ cd manual-2  
$ ls -al
```

서로 다른 파일 병합하기 - (2)

3. vim에서 work.txt 파일을 만들고 '1'을 입력한 후 저장하고 'work 1'이라는 메시지와 함께 커밋

```
$ vim work.txt  
$ git add work.txt  
$ git commit -m "work 1"
```

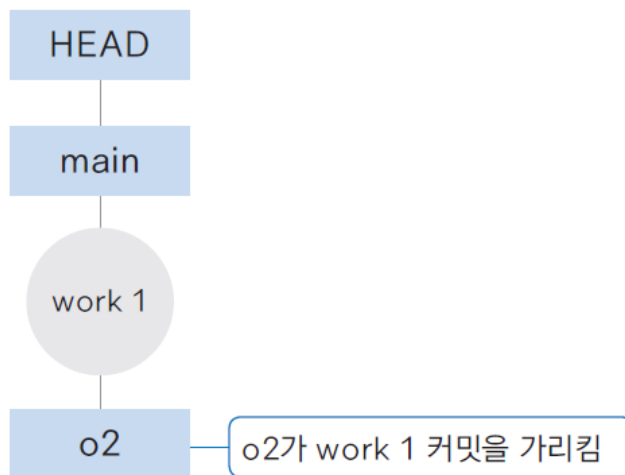
← work.txt를 만들고 '1' 입력 후 저장



서로 다른 파일 병합하기 - (3)

4. 이제 'o2'라는 브랜치를 만들

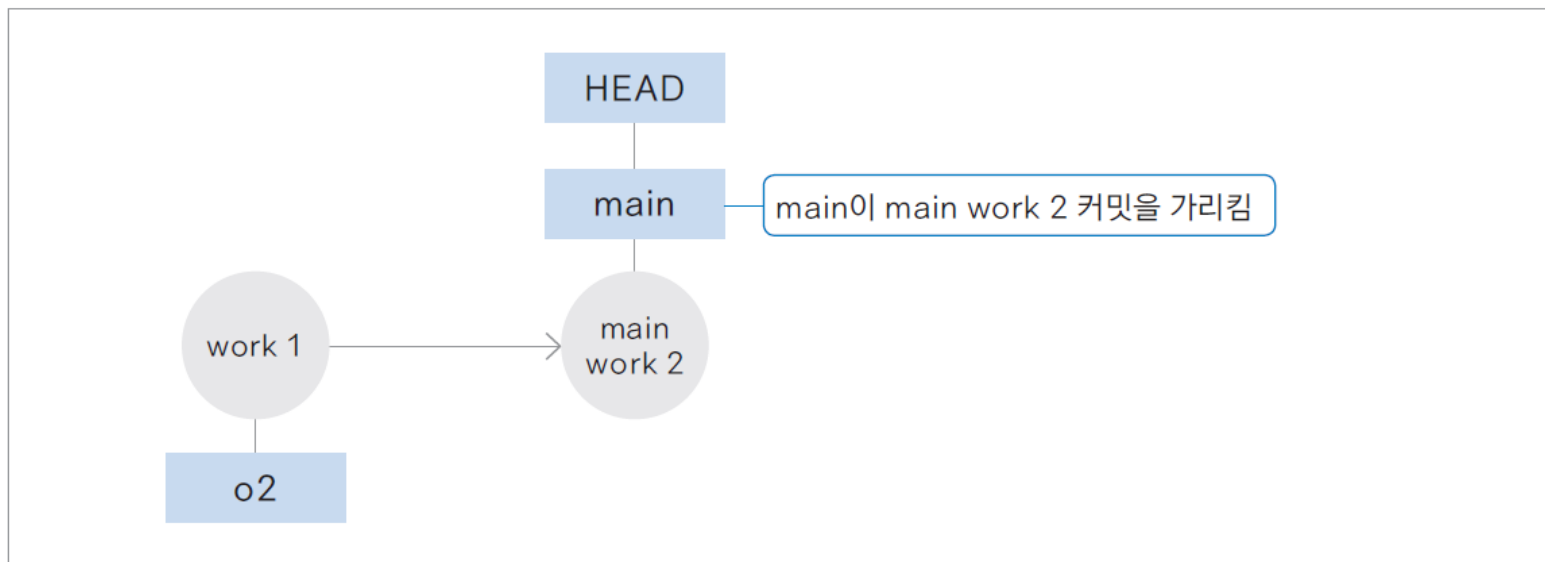
```
$ git branch o2
```



서로 다른 파일 병합하기 - (4)

5. main 브랜치에 main.txt라는 파일을 하나 더 만들
빔에서 'main 2'라고 입력한 후 저장하고 'main work 2'라는 메시지와 함께 커밋

```
$ vim main.txt    ← main.txt를 열고 'main 2' 입력 후 저장  
$ git add main.txt  
$ git commit -m "main work 2"
```

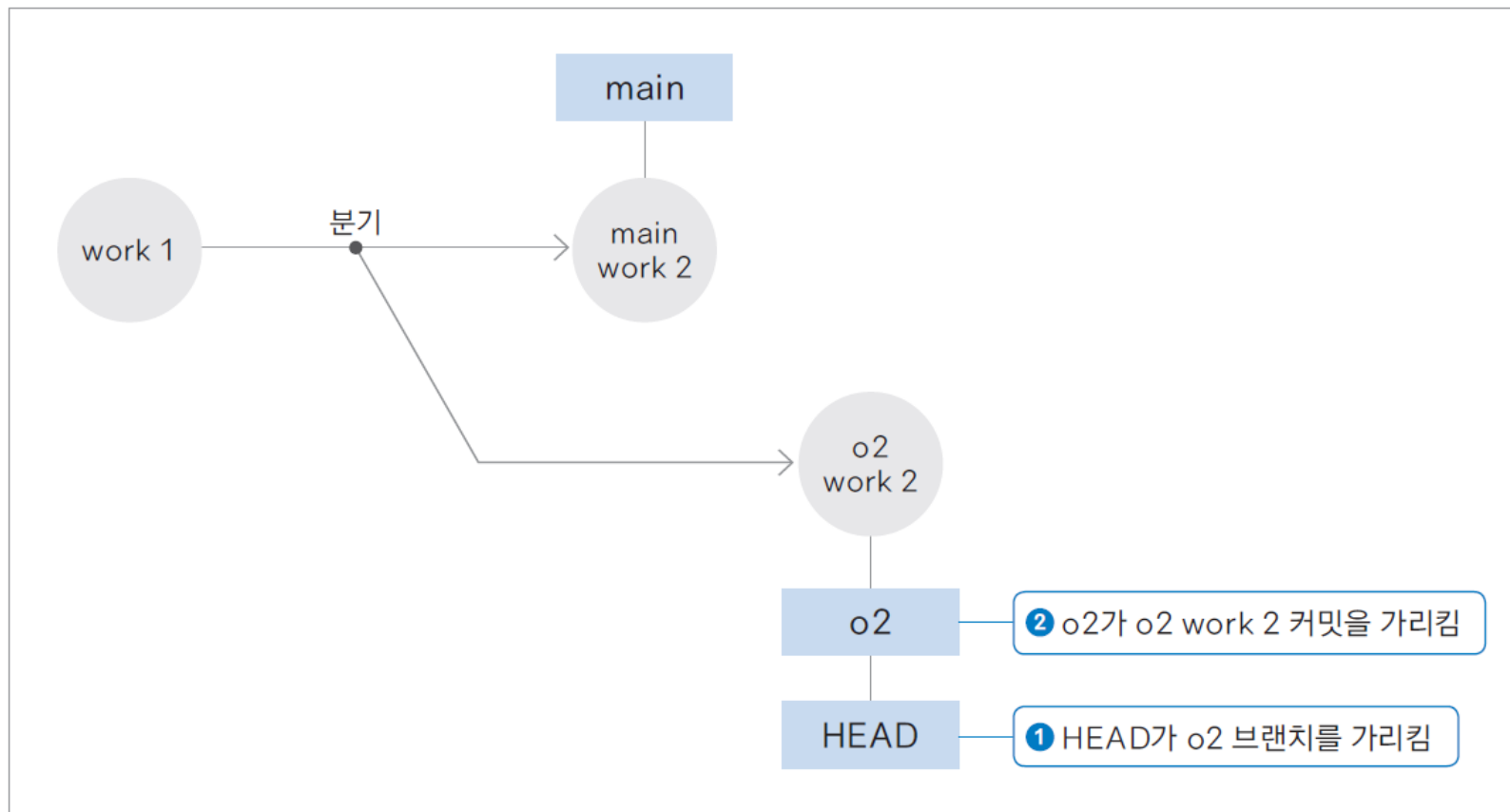


서로 다른 파일 병합하기 - (5)

6. 지금까지 main 브랜치에서 work 1 커밋과 main work 2 커밋을 만들었음
이제 o2 브랜치로 전환
7. o2 브랜치에서 o2.txt라는 파일을 만들고, '2'라는 내용을 입력한 후 저장
이어서 o2.txt 파일은 'o2 work 2'라는 메시지와 함께 커밋

```
$ vim o2.txt      ← o2.txt를 만들고 '2' 입력 후 저장
$ git add o2.txt
$ git commit -m "o2 work 2"
```

서로 다른 파일 병합하기 - (6)



서로 다른 파일 병합하기 - (7)

8. 현재 커밋의 상태를 확인하기 위해 git log 명령을 사용

work 1 커밋은 main 브랜치와 o2 브랜치가 똑같이 가지고 있음

이어서 main 브랜치에는 main work 2 커밋이 생겼고 o2 브랜치에는 o2 work 2 커밋이 생겼음

main 브랜치 입장에서 main work 1 → main work 2 순으로 커밋이 생겼고

o2 브랜치 입장에서 main work 1 → o2 work 2 순으로 커밋이 생겼음

```
$ git log --oneline --branches --graph
```

9. 이제 o2 브랜치에서 작업이 모두 끝났다고 가정하면 o2 브랜치의 내용을 main 브랜치로 병합해야 함
브랜치를 병합하려면 먼저 main 브랜치로 전환해야 함

```
$ git switch main
```

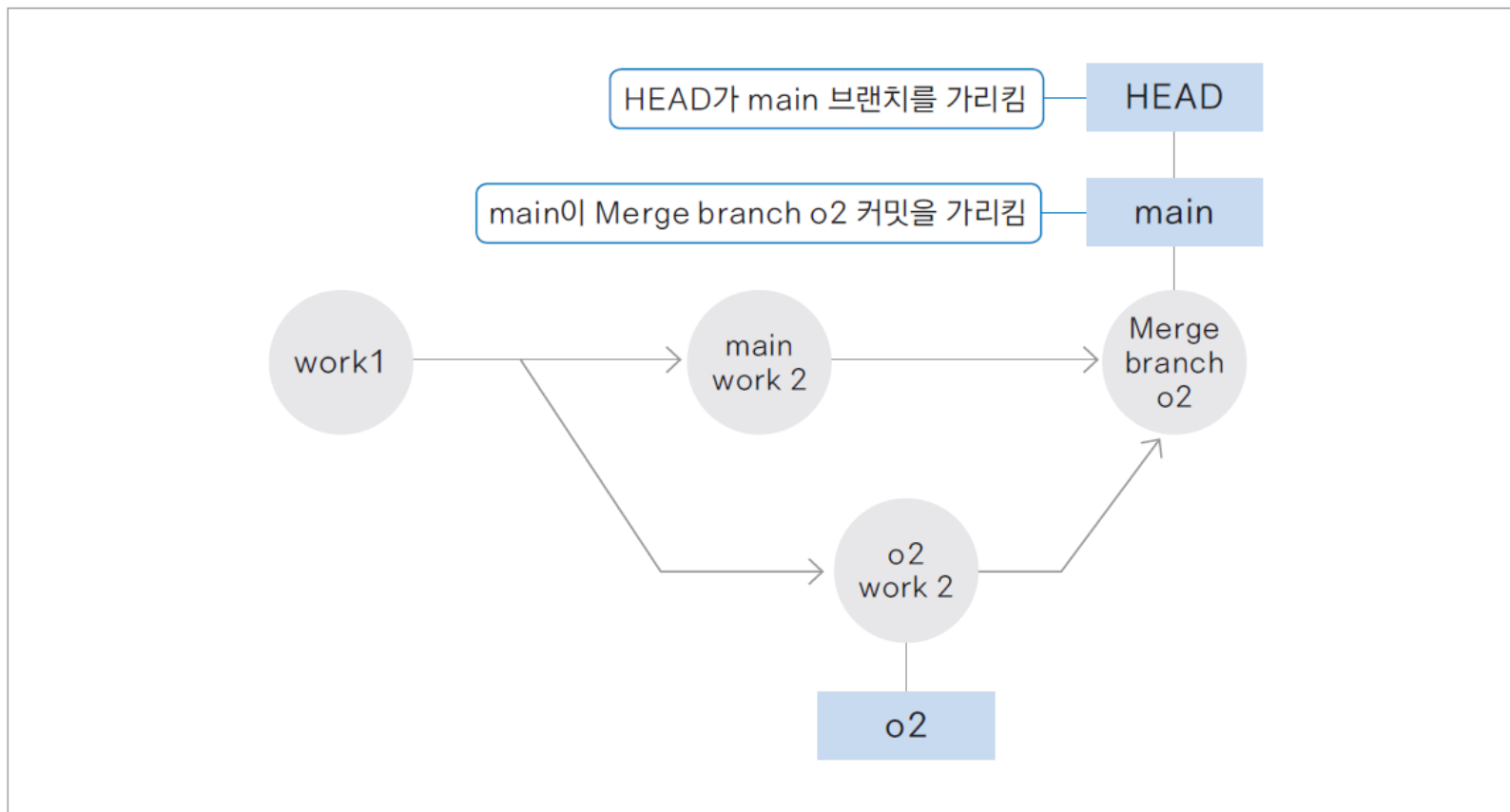
서로 다른 파일 병합하기 - (8)

10. 브랜치를 병합할 때는 git merge 명령 뒤에 가져올 브랜치 이름을 적음
main 브랜치를 기준으로 o2 브랜치를 가져와 병합하려면 다음과 같이 입력

```
$ git merge o2
```

11. vim이 자동으로 실행되면서 'Merge branch o2'라는 커밋 메시지가 나타남
브랜치를 병합하면서 만들어진 커밋 메시지인데 커밋 메시지를 수정할 수도 있고 그대로 사용해도 됨
{{ Esc }}를 누르고 :wq를 입력해 내용을 저장한 후 편집기를 종료

서로 다른 파일 병합하기 - (9)



서로 다른 파일 병합하기 – (10)

12. ls -al 명령을 사용해 확인해 보면 o2 브랜치에 있던 o2.txt 파일이 main 브랜치에 합쳐짐
13. git log 명령으로 브랜치와 커밋이 어떻게 병합되었는지 확인할 수도 있음
o2 work 2 커밋이 main 브랜치에 병합되면서 'Merge branch o2'라는 커밋이 새로 생겼음
두 브랜치에서 서로 다른 파일을 병합하는 경우 이렇게 깃에서 간단히 해결할 수 있음

```
$ git log --oneline --branches --graph
```

서로 다른 브랜치에서 한 문서의 다른 부분을 수정했을 때 병합하기 - (1)

1. 홈 디렉터리로 이동한 후 'manual-3'이라는 깃 저장소를 만들고, manual-3 디렉터리로 이동

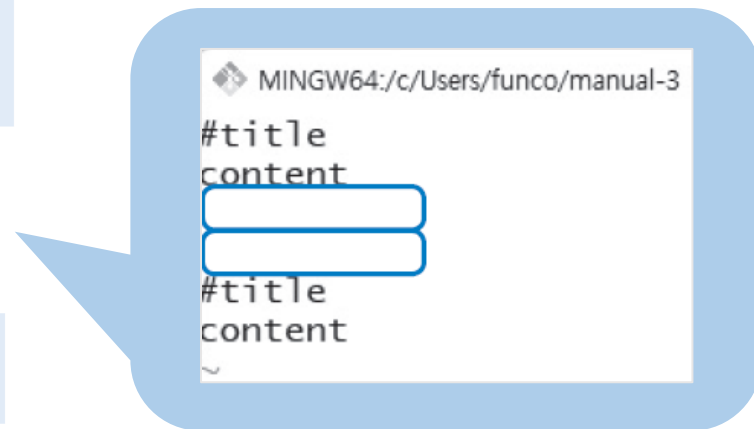
```
$ cd ~  
$ git init manual-3  
$ cd manual-3
```

2. work.txt 파일을 만들고 오른쪽 그림과 같이 입력
나중에 work.txt 문서를 수정하고 병합할 것이므로 중간에 공백을 두 줄 뒀다

```
$ vim work.txt
```

3. 방금 만든 work.txt를 스테이지에 올리고 커밋
커밋 메시지는 'work 1'이라고 함

```
$ git add work.txt  
$ git commit -m "work 1"
```



서로 다른 브랜치에서 한 문서의 다른 부분을 수정했을 때 병합하기 – (2)

4. 'o2'라는 새로운 브랜치를 만들

이제 main 브랜치와 o2 브랜치에는 모두 work 1 커밋이 있게 됨

```
$ git branch o2
```

5. 양쪽 브랜치 모두 work.txt가 있는 상태에서 우선 main 브랜치에서 문서를 수정하기 위해 vim 편집기에서 work.txt 문서를 열

```
$ vim work.txt
```

6. work.txt 문서에는 #title과 content가 각각 2개씩 있음

그중에서 첫 번째 content 다음 줄에 'main content 2'라고 입력한 후 저장

7. 수정한 work.txt를 커밋하고 커밋 메시지는 'main work 2'라고 함

```
$ git commit -am "main work 2"
```


서로 다른 브랜치에서 한 문서의 다른 부분을 수정했을 때 병합하기 – (3)

8. 이제 o2 브랜치의 work.txt 파일도 수정하기 위해 먼저 o2 브랜치로 전환하고 work.txt 파일을 불러옴

```
$ git switch o2  
$ vim work.txt
```

9. 이번에는 두 번째 content 다음 줄에 'o2 content 2'라고 입력하고 저장

10. 수정한 work.txt 파일을 커밋하고 커밋 메시지는 'o2 work 2'라고 함

```
$ git commit -am "o2 work 2"
```

11. main 브랜치와 o2 브랜치 양쪽에서 work.txt 파일을 수정했지만 문서 안의 수정 위치는 다름
이 상태에서 병합하기 위해 o2 브랜치를 main 브랜치에 합치기 위해 main 브랜치로 전환

```
$ git switch main
```

서로 다른 브랜치에서 한 문서의 다른 부분을 수정했을 때 병합하기 - (4)

12. git merge 명령을 사용해 o2 브랜치를 main 브랜치로 끌어옴

```
$ git merge o2
```

13. vim이 자동으로 실행되면서 커밋 메시지가 나타남
메시지를 수정할 수도 있고 그대로 사용할 수도 있음
원하는 메시지를 작성하고 저장한 후 편집기를 종료

14. 터미널 창에는 Auto-merging work.txt로 시작하는 병합 완료 메시지가 나타남
o2 브랜치의 work.txt가 main 브랜치의 work.txt와 어떻게 병합되었는지 보려면 cat 명령을 사용해 work.txt 파일을 확인

```
$ cat work.txt
```

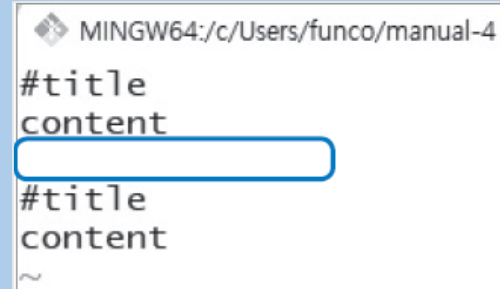
15. main 브랜치의 수정 내용과 o2 브랜치의 수정 내용이 자연스럽게 하나의 파일에 합쳐진 것을 볼 수 있음
이렇게 같은 문서로 서로 다른 위치를 수정했을 경우 자동으로 합쳐 주는 기능이 있어 깃은 더욱 강력한 도구가 됨

서로 다른 브랜치에서 한 문서의 같은 부분을 수정했을 때 병합하기 - (1)

1. 홈 디렉터리로 이동한 후 'manual-4'라는 깃 저장소를 만들고, manual-4 디렉터리로 이동

```
$ cd ~  
$ git init manual-4  
$ cd manual-4
```

2. vim에서 work.txt 파일을 만들고 오른쪽 그림과 같이 입력
나중에 work.txt 문서의 같은 위치를 수정하기 위해 두 내용 사이에 빈 줄을 하나만 둠



```
MINGW64:/c/Users/funco/manual-4  
#title  
content  
#title  
content  
~
```

3. 방금 만든 work.txt를 스테이지에 올리고 커밋하고 커밋 메시지는 'work 1'이라고 함

```
$ git add work.txt  
$ git commit -m "work 1"
```

서로 다른 브랜치에서 한 문서의 같은 부분을 수정했을 때 병합하기 - (2)

- 이제 o2라는 브랜치를 만들면 o2 브랜치는 main 브랜치의 최신 커밋을 가져옴
즉, 브랜치 양쪽에 work.txt가 있음

```
$ git branch o2
```

- 현재 브랜치인 main 브랜치에서 work.txt를 수정

```
$ vim work.txt
```

- work.txt 문서에는 #title과 content가 각각 2개씩 있음
첫 번째 content 다음 줄에 'main content 2'라고 입력한 후 저장

- 수정한 work.txt를 커밋하고 커밋 메시지는 'main work 2'라고 함

```
$ git commit -am "main work 2"
```

서로 다른 브랜치에서 한 문서의 같은 부분을 수정했을 때 병합하기 – (3)

8. 이제 o2 브랜치로 전환하고 work.txt 파일을 엽

```
$ git switch o2  
$ vim work.txt
```

9. main 브랜치에서 추가 내용을 입력했던 위치와 같은 곳에 'o2 content 2'라고 입력하고 저장

10. 수정한 work.txt 파일을 커밋하고 커밋 메시지는 'o2 work 2'라고 함

```
$ git commit -am "o2 work 2"
```

11. main 브랜치와 o2 브랜치 양쪽에서 work.txt 파일을 수정했는데 공교롭게도 문서 안의 수정 위치가 같음

o2 브랜치를 main 브랜치에 병합하기 위해 먼저 main 브랜치로 전환한 후, git merge 명령을 사용해 o2 브랜치를 병합

```
$ git switch main  
$ git merge o2
```

서로 다른 브랜치에서 한 문서의 같은 부분을 수정했을 때 병합하기 - (4)

12. 이전에 git merge 명령을 실행했을 때처럼 vim이 자동으로 열리지 않고 경고 메시지가 나타남
이 메시지는 work.txt를 자동 병합하는 동안 충돌이 발생했다는 뜻
13. 충돌이 생긴 문서는 자동으로 병합될 수 없으므로 사용자가 충돌 부분을 직접 해결한 후 커밋해야 함
충돌이 생긴 work.txt는 어떻게 되어 있는지 확인하기 위해 vim에서 work.txt를 열어 봄

```
$ vim work.txt
```

14. main 브랜치에서 수정한 내용과 o2 브랜치에서 수정한 내용이 한꺼번에 나타남
<<<<<< HEAD와 가운데 가로줄(=====) 사이의 내용은 현재 브랜치, 즉 main 브랜치에서 수정한 내용
가로줄(=====)과 >>>>>> o2 사이의 내용은 o2 브랜치에서 수정한 내용
양쪽 브랜치의 내용을 참고하면서 내용을 직접 수정해야 함
15. {{ A }}를 눌러 work.txt의 내용을 원하는 형태로 수정함
수정이 끝나면 문서에 있던 <<<<<< HEAD나 >>>>>> o2, 가로줄(=====)을 삭제
문서를 저장하고 편집기를 종료

서로 다른 브랜치에서 한 문서의 같은 부분을 수정했을 때 병합하기 - (5)

16. 이제 수정한 work.txt를 스테이지에 올리고 커밋하면 됨

커밋 메시지는 merge o2 branch로 함

이렇게 해서 o2 브랜치에서 병합한 work.txt의 충돌을 해결하고 커밋을 끝냈음

```
$ git commit -am "merge o2 branch"
```

17. git log 명령을 사용하면 지금까지 만든 브랜치와 커밋의 관계를 한눈에 확인

```
$ git log --oneline --branches --graph
```

병합이 끝난 브랜치 삭제하기 - (1)

1. git branch 명령을 사용하면 현재 저장소에 어떤 브랜치가 있는지 확인할 수 있음
앞의 브랜치 병합 실습을 따라왔다면 지금 위치는 manual-4 디렉터리일 것이고, 여기에는 main 브랜치와 o2 브랜치가 있음
현재 브랜치는 main이므로 main 브랜치 앞에 * 표시가 되어 있음

```
$ git branch
```

2. 저장소의 기본 브랜치는 main이므로 브랜치를 삭제하려면 main 브랜치에서 해야 함
현재 브랜치가 main 브랜치가 아니라면 main 브랜치로 전환

```
$ git switch main
```

3. 브랜치를 삭제할 때는 git branch 명령에 -d 옵션을 추가한 후 그 뒤에 삭제할 브랜치 이름을 사용
다음과 같이 o2 브랜치를 삭제

```
$ git branch -d o2
```


병합이 끝난 브랜치 삭제하기 - (2)

4. Deleted branch o2처럼 메시지가 나타나면 브랜치 삭제를 성공한 것

삭제한 브랜치는 같은 이름으로 다시 브랜치를 만들면 예전에 작업했던 내용이 그대로 나타남

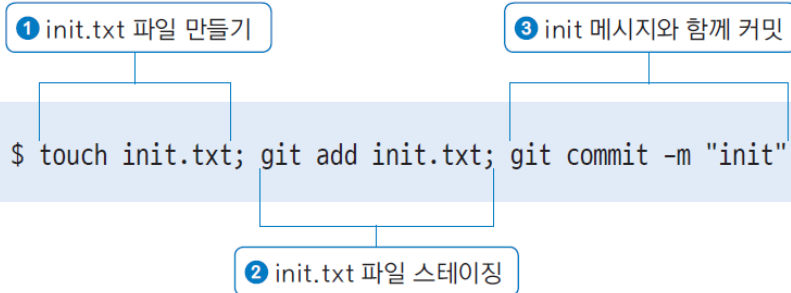
즉, 브랜치를 삭제한다는 것은 저장소에서 완전히 없애는 것이 아니라 깃의 흐름 속에서 감추는 것이라고 생각하면 됨

cherry-pick으로 병합하기 - (1)

1. 홈 디렉터리에 cherry-pick 디렉터를 깃 저장소로 만들고, cherry-pick 디렉터리로 이동

```
$ cd ~  
$ git init cherry-pick  
$ cd cherry-pick
```

2. 우선 새로운 브랜치를 만들기 전에 버전을 만들기 위해 init.txt 파일을 만든 후 스테이징해서 'init'이라는 메시지와 함께 커밋
터미널에서는 세미콜론(;)으로 구분해서 여러 개의 명령을 한꺼번에 처리할 수 있음
그리고 명령어 중 touch는 빈 파일을 만들
우리는 파일 내용을 수정하는 것이 아니라 버전이 필요하기 때문에 빈 파일을 만들고 스테이징과 커밋을 진행



```
$ touch init.txt; git add init.txt; git commit -m "init"
```

1 init.txt 파일 만들기

2 init.txt 파일 스테이징

3 init 메시지와 함께 커밋

cherry-pick으로 병합하기 - (2)

3. 이 상태에서 새로운 브랜치 topic을 만들

그리고 깃 로그를 확인해 보면 init이라는 버전이 main과 topic 브랜치 양쪽에 있는 것을 볼 수 있음
여기에서 --all 옵션은 최신 커밋 뿐만 아니라 모든 커밋을 다 보여주기 위한 옵션

```
$ git branch topic  
$ git log --oneline --all --graph
```

4. 이제 main 브랜치에 m1과 m2라는 2개의 버전을 더 만들고 git log로 확인

main 브랜치는 최신 버전 m2를 가리키고 있고, topic 브랜치에는 init 버전 상태인 것을 볼 수 있음

```
$ touch m1; git add m1; git commit -m "m1"  
$ touch m2; git add m2; git commit -m "m2"  
$ git log --oneline --all --graph
```

cherry-pick으로 병합하기 – (3)

5. topic 브랜치로 전환한 후, 앞에서와 같은 방법으로 t1, t2, t3 버전을 만들

```
$ git switch topic  
$ touch t1; git add t1; git commit -m "t1"  
$ touch t2; git add t2; git commit -m "t2"  
$ touch t3; git add t3; git commit -m "t3"  
$ git log --oneline --all --graph
```

6. git log로 확인해 보면 main 브랜치는 init 버전부터 시작해서 m1, m2 버전으로 연결되어 있고, topic 브랜치는 init 버전부터 시작해서 t1, t2, t3 버전으로 연결

cherry-pick으로 병합하기 - (4)

- 현재 2개의 브랜치가 있는데, topic 브랜치의 t2 버전에서 적용했던 내용을 main 브랜치에도 적용하고 싶을 때 사용하는 것이 바로 cherry-pick으로 topic 브랜치의 여러 버전 중 하나를 골라와서 병합하는 것
cherry-pick을 사용하려면 우선 병합될 브랜치인 main 브랜치로 전환해야 함

```
$ git switch main
```

- topic 브랜치의 t2 버전만 병합하려면, cherry-pick 명령 다음에 t2에 해당하는 커밋 해시를 알려주면 됨

```
$ git cherry-pick 체리픽할 해시
```

- cherry-pick 후에 main 브랜치에 어떤 파일이 있는지 살펴보면
기존에 main 브랜치에 있던 init.txt, m1, m2 다음에 t2가 합쳐진 것을 볼 수 있음

```
$ ls
```

- 그리고 깃 로그를 확인해 보면, topic 브랜치에서 t2가 있고, main 브랜치에도 t2가 추가되어 있음